Inserts a fence with the required memory-ordering constraints. This is equivalent to `std::atomic_thread_fence(order)` except that the constraints apply only between a thread and a signal handler on the same thread.

Throws

Nothing.

D.3.7. std::atomic_flag class

The `std::atomic_flag` class provides a simple bare-bones atomic flag. It's the only data type that's *guaranteed* to be lock-free by the C++11 Standard (although many atomic types will be lock-free in most implementations).

An instance of `std::atomic_flag` is either *set* or *clear*.

Class definition

```
struct atomic_flag
{
    atomic_flag() noexcept = default;
    atomic_flag(const atomic_flag&) = delete;
    atomic_flag& operator=(const atomic_flag&) = delete;
    atomic_flag& operator=(const atomic_flag&) volatile = delete;

    bool test_and_set(memory_order = memory_order_seq_cst) volatile
     noexcept;
    bool test_and_set(memory_order = memory_order_seq_cst)
noexcept;
    void clear(memory_order = memory_order_seq_cst) volatile
noexcept;
    void clear(memory_order = memory_order_seq_cst) noexcept;
};

bool atomic_flag_test_and_set(volatile atomic_flag*) noexcept;
bool atomic_flag_test_and_set(atomic_flag*) noexcept;
bool atomic_flag_test_and_set_explicit(
    volatile atomic_flag*, memory_order) noexcept;
bool atomic_flag_test_and_set_explicit(
    atomic_flag*, memory_order) noexcept;
```